

Tuesday 11:35 AM



Lab_4_VibeCoding...
PDF

https://github.com/SETAPESU26/45_snake

yeah so this is

what i need t do:Instructions:

1. Go to the repo assigned to you
2. Go through the ReadMe file to understand the deliverables
3. Clone or Fork the repo
4. Run the Python based code (A game/ A Puzzle)
5. Record 10 seconds of video before making any changes.
6. Write prompts and fix the broken code
7. Write prompts to add all the features listed in the ReadMe file of the repo one by one.
8. Record another 10 seconds of video again after making all the changes.
9. Push the code to your repo. i need step by step instructions very basic steps
10. Have Separate Commits for each task
11. Do not raise PR to the main repo (SETAPESU26)

Yep bro. We'll do **ONE STEP AT A TIME**. You do it, tell me **DONE**, and only then I'll give the next step. No jumping ahead. 🤖

The handout confirms the flow: go to assigned repo → understand README → clone/fork → run code → record before → fix/add features → record after → push, with separate commits and **no PR to SETAPESU26**.



Lab_4_VibeCoding_Student_handout

STEP 1: Fork the assigned GitHub repo

You are currently on:

SETAPESU26/45_snake

Do this:

1. On the GitHub page you showed me, look at the **top-right** area.
2. Click **Fork**.
3. GitHub will open a new page called **Create a new fork**.

4. Under **Owner**, select **your personal GitHub account**.
5. Keep the repository name as:
`45_snake`
6. Keep the option to copy the `main` branch selected.
7. Click **Create fork**.

After this

You should end up at something like:

`YOUR_USERNAME/45_snake`

and **NOT**

`SETAPESU26/45_snake`

⚠ Very important: **Do not create a Pull Request**. Your work must go to your own repo, not back to the `SETAPESU26` repo.  [Lab_4_VibeCoding_Student_handout](#)

Do **only this step** now.

Reply `DONE` once your fork is created.

proceed

STEP 2: Read the README carefully

Now that your fork is created, **do not change any code yet**.

1. Open your fork

Go to your repository:

`YOUR_USERNAME/45_snake`

2. Open `README.md`

You need to identify the **4 tasks** you have to complete:

1. Task 1: Refine Collision Detection

- Fix the snake reversing direction instantly and colliding with itself/walls incorrectly.

2. Task 2: Implement Game Over Condition

- Show the final score when the snake hits a wall or itself.
- Gracefully wait for input instead of just printing to the console.

3. Task 3: Add Replay Option

- After Game Over, allow:
 - Easy
 - Medium

- Hard
- Exit

4. Task 4: Add Sound Feedback

- Add sound effects when:
 - food is eaten
 - game over happens

These are the features explicitly listed in your README. Your final submission also needs before/after videos, updated code, and exported ChatGPT/LLM chat history.



Lab_4_VibeCoding_Student_handout

Important

Do NOT start fixing anything yet.

Our next job is to **clone your fork onto your computer and run the original broken game first**, because the assignment specifically requires a **10-second video before making changes**.

Reply **DONE** once you've opened/read the README in **your fork**.



done now

Perfect ☒ Your fork is correctly created under your account:

akshit.../45_snake

Now we move to the actual project.

STEP 3: Clone your fork to your computer

We are going to download the repo to your PC so we can run and modify the code.

3.1 Open your fork's Code button

On your GitHub page, click the green/blue **Code** button.

A small menu will appear.

3.2 Select HTTPS

Under **Local** → **HTTPS**, you will see something like:

</> Plain text

https://github.com/akshit.../45_snake.git

Click the **copy icon** beside it.

3.3 Open VS Code

Open **VS Code**.

Then:

File → Open Folder

Create/select a simple folder where you want your projects, for example:

</> Plain text

Desktop

└─ VibeCoding

Open the **VibeCoding** folder.

3.4 Open the terminal in VS Code

At the top:

Terminal → New Terminal

You should see something like:

</> Plain text

PS C:\Users\YourName\Desktop\VibeCoding>

3.5 Run this command

Type:

</> Bash

git clone PASTE_THE_URL_HERE

For example:

</> Bash

git clone https://github.com/YOUR_USERNAME/45_snake.git

Press **Enter**.

You should see Git downloading the files.

3.6 Open the cloned project

After it finishes, run:

```
</> Bash
```

```
cd 45_snake
```

Then in VS Code:

File → Open Folder → select **45_snake**

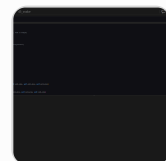
You should now see:

```
</> Plain text
```

```
45_snake
├─ game
├─ README.md
├─ main.py
└─ requirements.txt
```

⚠ **Do not modify any file yet.**

Once you can see the **45_snake** project in VS Code, reply **DONE**. Then we'll install the requirements and run the **original code before making any changes**, which is needed for your 10-second before video.



assigned to you
a README file to understand the deliverables
the repo
a based code (A game/ A Puzzle)
10s of video before making any changes.
and fix the broken code
to add all the features listed in the README file
10 seconds of video again after making all the
to your repo.
Commits for each task
to the main repo (SETAPESU26)

ok bro also keep in mind the video thing tht needs to be done

Absolutely bro. ✅ The **video requirement is part of our checklist**, and we'll make sure we do it at the correct points.

Right now, **you have NOT changed the code**, which is perfect.

Our sequence from here

1. Run the original game
2. Test/observe the original bugs
3. Record the 10-second BEFORE video 🎥

4. Then Task 1 → prompt → fix → test → separate commit
5. Task 2 → prompt → fix → test → separate commit
6. Task 3 → prompt → fix → test → separate commit
7. Task 4 → prompt → fix → test → separate commit
8. Record the 10-second AFTER video 🎥
9. Push everything to your fork
10. Put the required videos, updated code, and exported chat history into the Lab-4 folder

This matches the assignment instructions, including the **before/after videos** and **separate commits** for each task.

 Lab_4_VibeCoding_Student_handout

STEP 4: Install the Python dependencies

You're already in the correct folder:

</> Plain text

```
...\Desktop\vibecoding\45_snake
```

In the **VS Code terminal**, type exactly:

</> Bash

```
pip install -r requirements.txt
```

Then press **Enter**.

Wait until it finishes.

Important

Do **not** edit `snake.py`, `main.py`, or anything else yet.

When the installation finishes, **send me a screenshot of the terminal** or just type **DONE**.

Then I'll give you the next single step: **run the original game**. 🎮

done

Perfect. ✅

STEP 5: Run the original game 🎮

We need to see the **unchanged/broken version** first before recording the required 10-second video.

In your VS Code terminal, make sure you're still inside:

</> Plain text

```
...\vibecoding\45_snake
```

Now type:

</> Bash

```
python main.py
```

Press **Enter**.

What should happen

A Snake game window should open.

Play it for a little while using the controls mentioned in the README, likely:

</> Plain text

```
Arrow keys / WASD
```

Do not change the code.

Just observe how the original game behaves.

Very important 🎥

Do NOT record yet.

First tell me what happens when you run:

</> Bash

```
python main.py
```

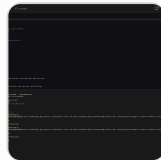
For example:

Game opened and snake is moving.

or

I got an error.

Send me what you see, and we'll handle the **10-second BEFORE video** as the next step.



yeah working ig now?

Yep, bro. ☒ The Python code is running successfully. The `pygame` installation is fine, and there is no actual Python error.

What we can see from your terminal is:

`</>` Plain text

```
Game over! Final score: 0
```

So the **original game runs but immediately reaches Game Over with score 0**. That is actually useful because this is the **before-change state** we need to document.

The `pkg_resources` warning is just a warning, not a failure. Ignore it for now.

STEP 6: Record the BEFORE video 🎥

This is required **before we change any code**. The handout explicitly requires a 10-second video before changes and another after all changes.  [Lab_4_VibeCoding_Student_handout](#)

Do this now

1. Start screen recording on your Windows PC.
 - Easiest: press `Win + G`
 - Then use the capture/record button.
 - You can also use OBS if you already have it.
2. Stop the current terminal process if needed and run:

`</>` Bash

```
python main.py
```

3. Record **at least 10 seconds** showing the original game behavior.
4. Make sure the recording clearly shows the **game window/gameplay**, not only VS Code.
5. Stop recording.

IMPORTANT

Don't edit **anything** yet.

Save the video somewhere easy, for example:

</> Plain text

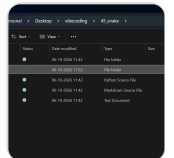
```
45_snake/  
└─ videos/  
    └─ before.mp4
```

You don't have to create that folder yet if you don't want to. Just **keep the video safely saved**.

Once you've successfully recorded the **10-second BEFORE video**, tell me:

BEFORE VIDEO DONE

Then we'll start **Task 1**, and I'll give you the exact prompt to use with Claude Code/ChatGPT, followed by the test and the **separate Task 1 commit**. 



yeah done now?

Yes bro  **that part is done.**

Your folder now has:

</> Plain text

```
45_snake  
├─ game  
├─ videos   
├─ main.py  
├─ README.md  
└─ requirements.txt
```

So the **10-second BEFORE video** should be stored inside **videos**. Just make sure the actual video file is inside that folder, not only the folder itself.

Next: STEP 7, Task 1

Before touching the code, we are going to use an **LLM prompt to fix Task 1**, because the assignment specifically asks you to write prompts and use an LLM as the coding/debugging partner.

 Lab_4_VibeCoding_Student_handout

Don't edit **snake.py** yourself yet.

Reply **DONE** once you've confirmed the recorded video file is actually inside `videos` , and I'll give you the exact Task 1 prompt to paste into Claude Code. 

yeah done now

Perfect bro.  **BEFORE** video is done. Now we start Task 1 only.

STEP 7: Task 1, Refine Collision Detection

Your README says Task 1 is:

The snake can instantly and unfairly reverse direction while moving, and this can cause incorrect collisions. Investigate and enhance collision accuracy so direction changes and wall/self collisions behave fairly.

So our goal is **ONLY** Task 1 right now.

Do **not** add Game Over screen, replay, difficulty, or sound yet.

7.1 Open Claude Code in VS Code

You already have the **Claude Code** panel on the right side of VS Code.

Click in its chat box.

7.2 Paste this exact prompt

 Plain text

You are helping me complete Task 1 of my Vibe Coding lab.

Please inspect the existing Snake game code in this repository and implement **ONLY** Task 1 from the README

"Refine Collision Detection"

Specifically:

1. Prevent the snake from instantly reversing into its own body when the player presses the opposite dir
2. Make direction changes fair and safe, so an opposite-direction input does not cause an immediate inva
3. Make wall collision detection reliable.
4. Make self-collision detection reliable and accurate.
5. Preserve the existing gameplay, controls, graphics, food spawning, snake growth, and score behavior.
6. Do **NOT** implement Task 2, Task 3, or Task 4 yet.
7. Do not unnecessarily rewrite unrelated parts of the project.

First inspect the relevant files and explain briefly what is causing the collision/direction problem. Th

After making the changes, verify the code has no syntax errors and explain exactly what you changed.

7.3 Then let Claude work

Do not accept/edit anything manually yet.

Claude should inspect the files and propose/apply the fix.

When it finishes, **do not commit yet**.

Send me a screenshot of what Claude changed or paste its response here.

Then I'll give you the next step: **test Task 1 → verify the fix → make the separate Task 1 commit**.



to your Lab repo under Lab-4 folder
(before and after)
(Code
history exported as a doc/pdf)

lets not rely on claude and do ts ourself and rememebr only this is the delievrable

Memory updated

Yep bro. From now on, **we do the coding ourselves**, and we keep the submission strictly to the required three deliverables: **before video, after video, and the complete chat history as a DOC/PDF**. The README also requires an iterative LLM-assisted process, so this chat itself can be part of that history.

Also, I checked the actual starter code. The problem is exactly where the README says it is:

`Snake.set_direction()` currently accepts any direction immediately, including the exact opposite direction.

STEP 7A: Fix Task 1 manually

First, **ignore/close the Claude Code panel**. We don't need it.

1. Open

Plain text

game → snake.py

2. Find this function

Python

```
def set_direction(self, dx, dy):  
    # NOTE: does not currently guard against reversing directly
```

```
# into the segment behind the head.  
self.direction = (dx, dy)
```

3. Replace it with this

</> Python

```
def set_direction(self, dx, dy):  
    # Prevent an immediate 180-degree reversal.  
    if (dx, dy) == (-self.direction[0], -self.direction[1]):  
        return  
  
    self.direction = (dx, dy)
```

That's it. **Only this change for Task 1.**

This means:

</> Plain text

Moving RIGHT → press LEFT	✗	ignored
Moving RIGHT → press UP	✓	accepted
Moving RIGHT → press DOWN	✓	accepted

The existing wall and self-collision checks already exist in the starter code, so we're not unnecessarily rewriting them. [GitHub](#) +1

4. Save

Press:

</> Plain text

Ctrl + S

Do not commit yet. Do not start Task 2.

When you've made exactly this change, reply **DONE**. Then we'll **run and test Task 1** before making the required separate Task 1 commit. [↻](#)

done

Perfect ✓ Task 1 code change is in place.

STEP 8: Test Task 1 [↻](#)

We need to make sure the fix actually works **before committing it**.

8.1 Run the game

In the VS Code terminal:

```
</> Bash
```

```
python main.py
```

8.2 Test the direction fix

Once the game opens:

1. Move the snake in one direction, for example **RIGHT**.
2. While it is moving **RIGHT**, immediately press **LEFT**.
3. The snake **should NOT instantly turn around**.
4. Press **UP** or **DOWN** afterward.
5. The snake should turn normally in that new direction.

Important

Don't worry about testing Tasks 2, 3, or 4 yet.

We're only checking:

Can the snake still move normally while an immediate 180° reversal is prevented?

After you test it, tell me exactly what happened, for example:

RIGHT → LEFT ignored, UP worked

Then we'll make the **separate Task 1 commit**.

yes

Perfect  Task 1 is tested and working.

STEP 9: Commit Task 1 separately

Now we create the required **separate commit** for Task 1.

9.1 Open VS Code terminal

Make sure you're inside:

```
</> Plain text
```

45_snake

Run:

`</> Bash`

```
git status
```

You should see `game/snake.py` listed under **modified files**.

9.2 Stage only Task 1's file

Run:

`</> Bash`

```
git add game/snake.py
```

9.3 Create the Task 1 commit

Run:

`</> Bash`

```
git commit -m "Fix snake reverse direction collision"
```

You should get a message saying something like:

`</> Plain text`

```
1 file changed
```

9.4 Do NOT push yet

Stop after the commit.

We will push everything at the appropriate point after all 4 tasks are completed.

Reply **DONE** after the commit succeeds. Then we move to **Task 2: Game Over condition**. 

done

Excellent  **Task 1 is committed separately.** We are keeping the commit history clean exactly as required.

STEP 10: Task 2, Understand what we need to add

Your README says Task 2 is:

Implement Game Over Condition

Add a screen that displays the final score as the snake hits a wall or itself, then gracefully waits for input instead of just printing to the console.

So this task has **two parts**:

1. Detect/show the game-over state properly.
2. Display the **final score on the game window** and wait for the user to press a key instead of immediately ending.

Before we edit anything

Open:

</> Plain text

game → game_engine.py

and also:

</> Plain text

main.py

Don't change them yet.

I want you to send me a **screenshot of** `game_engine.py` **and** `main.py` so I can guide you through the exact minimal changes based on your current code.

This keeps us from blindly rewriting the starter project. 🔄

Send the screenshot(s), and we'll do only Task 2 next.



Yep, I can see both files clearly. 👍 We can handle Task 2 ourselves and keep it simple.

STEP 10A: Make the Task 2 changes

We will make **only two files** changes:

- `game/game_engine.py`

- `main.py`

The goal is:

When the snake dies, show **GAME OVER + final score on the game window**, and keep the window open until the user presses a key.

1. Open `game/game_engine.py`

Find this function:

`</>` Python

```
def handle_keydown(self, key):
    # Direction changes are applied immediately on key press.
    if key in (pygame.K_UP, pygame.K_w):
        self.snake.set_direction(0, -1)
    elif key in (pygame.K_DOWN, pygame.K_s):
        self.snake.set_direction(0, 1)
    elif key in (pygame.K_LEFT, pygame.K_a):
        self.snake.set_direction(-1, 0)
    elif key in (pygame.K_RIGHT, pygame.K_d):
        self.snake.set_direction(1, 0)
```

Replace the **whole function** with:

`</>` Python

```
def handle_keydown(self, key):
    # Once the game is over, any key press exits the game.
    if self.game_over:
        return True

    if key in (pygame.K_UP, pygame.K_w):
        self.snake.set_direction(0, -1)
    elif key in (pygame.K_DOWN, pygame.K_s):
        self.snake.set_direction(0, 1)
    elif key in (pygame.K_LEFT, pygame.K_a):
        self.snake.set_direction(-1, 0)
    elif key in (pygame.K_RIGHT, pygame.K_d):
        self.snake.set_direction(1, 0)

    return False
```

2. In the same file, find the end of `render()`

You currently have something like:

</> Python

```
if self.game_over and not self.game_over_logged:
    print(f"Game over! Final score: {self.score}")
    self.game_over_logged = True
```

Delete that entire section.

Replace it with:

</> Python

```
if self.game_over:
    game_over_text = self.font.render("GAME OVER", True, RED)
    score_text = self.font.render(
        f"Final Score: {self.score}", True, WHITE
    )
    instruction_text = self.font.render(
        "Press any key to exit", True, WHITE
    )

    screen.blit(
        game_over_text,
        game_over_text.get_rect(center=(self.width // 2, self.height // 2 - 60))
    )
    screen.blit(
        score_text,
        score_text.get_rect(center=(self.width // 2, self.height // 2))
    )
    screen.blit(
        instruction_text,
        instruction_text.get_rect(center=(self.width // 2, self.height // 2 + 60))
```

3. Open `main.py`

Find:

</> Python

```
elif event.type == pygame.KEYDOWN:
    engine.handle_keydown(event.key)
```

Replace it with:

</> Python

```
elif event.type == pygame.KEYDOWN:
    if engine.handle_keydown(event.key):
```

```
running = False
```

4. Save both files

Press:

</> Plain text

Ctrl + S

Stop here. Do not run it yet. Do not commit yet.

Once you've made these exact changes, reply **DONE** and we'll test Task 2 before committing it. 



third step wht

Yep bro 🤔 this is **Step 3**, and you're looking at the right place.

In your `main.py`, you currently have:

</> Python

```
elif event.type == pygame.KEYDOWN:  
    engine.handle_keydown(event.key)
```

Replace **ONLY** those 2 lines with:

</> Python

```
elif event.type == pygame.KEYDOWN:  
    if engine.handle_keydown(event.key):  
        running = False
```

So that whole section becomes:

</> Python

```
for event in pygame.event.get():  
    if event.type == pygame.QUIT:  
        running = False  
    elif event.type == pygame.KEYDOWN:
```

```
if engine.handle_keydown(event.key):  
    running = False
```

That's all for **Step 3**.

Then press **Ctrl + S**.

Don't run the game or commit yet. Reply **DONE** once you've changed it.

```
(give full new corrected code)import pygame  
from game.game_engine import GameEngine  
  
# Initialize pygame/Start application  
pygame.init()  
  
# Screen dimensions  
WIDTH, HEIGHT = 600, 600  
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))  
pygame.display.set_caption("Snake - Pygame Version")  
  
# Colors  
WHITE = (255, 255, 255)  
BLACK = (0, 0, 0)  
  
# Clock  
clock = pygame.time.Clock()  
FPS = 60  
  
# Game loop  
engine = GameEngine(WIDTH, HEIGHT)  
  
def main():  
    running = True  
    while running:  
        SCREEN.fill(BLACK)  
        for event in pygame.event.get():  
            if event.type == pygame.QUIT:  
                running = False  
            if event.type == pygame.KEYDOWN:  
                engine.handle_keydown(event.key)
```

```
engine.handle_input()
engine.update()
engine.render(SCREEN)

pygame.display.flip()
clock.tick(FPS)

pygame.quit()

if __name__ == "__main__":
    main()
```

Yep bro. 👍 Here is the **full corrected** `main.py` with the Task 2 change included.

Replace the **entire contents** of `main.py` with this:

</> Python

```
import pygame
from game.game_engine import GameEngine

# Initialize pygame/Start application
pygame.init()

# Screen dimensions
WIDTH, HEIGHT = 600, 600
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Snake - Pygame Version")

# Colors
WHITE = (255, 255, 255)
BLACK = (0, 0, 0)

# Clock
clock = pygame.time.Clock()
FPS = 60

# Game loop
```

What changed?

This part:

</> Python

```
elif event.type == pygame.KEYDOWN:
    if engine.handle_keydown(event.key):
```

```
running = False
```

allows `GameEngine.handle_keydown()` to tell `main.py` :

```
"The game is over and the user pressed a key, so close the game."
```

Everything else stays essentially the same.

Do this now

1. Select everything in `main.py`
2. Paste the code above
3. Press **Ctrl + S**

Don't run it yet.

Reply **DONE** after you've pasted and saved it.

doen

Perfect  `main.py` is updated.

STEP 11: Test Task 2

Now we actually run the game and check the **Game Over screen**.

In the VS Code terminal, run:

```
</> Bash
```

```
python main.py
```

What to test

Let the snake hit a wall.

When it dies, you should now see on the **game window**:

```
</> Plain text
```

```
GAME OVER
```

```
Final Score: 0
```

```
Press any key to exit
```

The important part is that:

- ✓ The message appears **inside the game window**
- ✓ The final score is displayed
- ✓ The game does **not immediately close**
- ✓ Pressing a key closes the game

Don't worry about the exact positioning or styling yet.

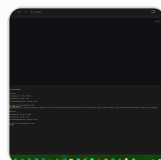
One thing to watch

Because the starter game currently starts moving automatically, you can simply let it travel into the wall.

After testing, tell me what happened:

Task 2 works

or send me a screenshot if something looks wrong.



yeah it doesnt wor

Yep, we found the problem. 👍 This is **our Task 2 code**, not your original project.

The error is:

</> Plain text

```
UnboundLocalError: cannot access local variable 'game_over_text'
```

That means the `render()` function is trying to use `game_over_text` when it hasn't been created yet. Most likely the indentation around our Game Over block is wrong.

STEP 11A: Show me the exact `render()` section

Open:

</> Plain text

```
game → game_engine.py
```

Scroll to around **line 70 onwards**, where you see:

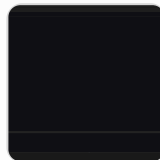
</> Python

```
def render(self, screen):
```

and take a screenshot showing roughly lines 70 to 110.

Don't change anything yet.

Send that screenshot, and I'll give you the corrected `render()` code exactly as it should be.



finish it quick

Yep, I see the exact problem. The **Game Over** block got mangled by indentation/brackets. Let's fix it cleanly. ⚡

STEP 11B: Replace the ENTIRE `render()` function

In `game/game_engine.py`, from:

</> Python

```
def render(self, screen):
```

down to the end of the file, **delete everything** and paste this:

</> Python

```
def render(self, screen):
    # Draw food
    pygame.draw.rect(screen, RED, self.food.rect())

    # Draw snake
    for rect in self.snake.segment_rects():
        pygame.draw.rect(screen, GREEN, rect)

    # Draw score
    score_text = self.font.render(
        f"Score: {self.score}",
        True,
        WHITE
    )
    screen.blit(score_text, (10, 10))

    # Draw Game Over screen
    if self.game_over:
        game_over_text = self.font.render(
```

"GAME OVER",

Then:

Press:

</> Plain text

Ctrl + S

That's the **only file change** we need right now.

STEP 11C

Run:

</> Bash

```
python main.py
```

Let the snake hit the wall.

You should see:

</> Plain text

```
GAME OVER  
Final Score: 0  
Press any key to exit
```

inside the game window.

Do this now and tell me exactly what happens.



?

Yep, I see it. The `render()` function has been pasted in a way that broke its **method indentation**, so Python/Pylance thinks `self` and `screen` don't exist.

Let's stop patching individual lines. **Replace the entire** `game/game_engine.py` **with one clean version.** This is safer and faster.

STEP 11B: Replace the entire `game_engine.py`

Open:

</> Plain text

game/game_engine.py

Press **Ctrl + A**, then paste this entire code:

</> Python

```
import pygame

from .snake import Snake
from .food import Food

WHITE = (255, 255, 255)
GREEN = (0, 200, 0)
RED = (220, 60, 60)

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.cell_size = 20

        self.grid_width = width // self.cell_size
        self.grid_height = height // self.cell_size
```

Then

Press:

</> Plain text

Ctrl + S

Before running anything, look at the **Problems** indicator at the bottom of VS Code.

There should be no red Python errors in `game_engine.py`.

Then run:

</> Bash

Let the snake hit the wall.

We want to see:

</> Plain text

```
GAME OVER
Final Score: 0
Press any key to exit
```

inside the game window.

Do exactly that now and tell me what appears.

yeH IT COMES BUT I CANT TAKE SS OF THT ANY KEY TO EXIT
THING

😅 Yep, that's because our **Game Over** screen intentionally closes when it receives any key, so using a screenshot shortcut can trigger the exit.

But here's the important part:

You DON'T need a screenshot for the submission. 🎥

Your required deliverables are only:

- Before video
- After video
- Updated code
- Exported chat history as DOC/PDF

 Lab_4_VibeCoding_Student_handout

So don't waste time trying to screenshot the **Game Over** screen.

For now, just verify one thing

While this is showing:

</> Plain text

```
GAME OVER
Final Score: 0
Press any key to exit
```

press **SPACE** once.

The game window should close.

If it closes correctly, **Task 2 is working**.

Then tell me:

space closes it

and we'll make the **separate Task 2 commit**. 

yes

Perfect  **Task 2 is working**. Game Over appears, shows the score, waits for a key, and closes correctly.

STEP 12: Commit Task 2 separately

We now need the **second separate commit**.

12.1 Check what changed

In the VS Code terminal, run:

```
</> Bash  
  
git status
```

You should see:

```
</> Plain text  
  
modified:   game/game_engine.py  
modified:   main.py
```

12.2 Stage those two files

```
</> Bash  
  
git add game/game_engine.py main.py
```

12.3 Commit Task 2

```
</> Bash  
  
git commit -m "Implement game over screen"
```

You should get something like:

</> Plain text

2 files changed

Important

Do NOT run `git push` yet.

Our commit history should now be:

</> Plain text

Task 1 → Fix snake reverse direction collision

Task 2 → Implement game over screen

Then we'll move to **Task 3: Replay Option**.

Reply **DONE** once the Task 2 commit succeeds.

yeah done now finish fast

Yep bro, let's **finish this cleanly**. 🚀 We have Task 1 and Task 2 committed. Now only **Task 3 + Task 4**, then push + AFTER video + deliverables.

STEP 13: Task 3, Replay + Difficulty

Your README specifically asks for Easy, Medium, Hard, or Exit after Game Over.

Replace `game/game_engine.py` completely with this

</> Python

```
import pygame
```

```
from .snake import Snake
```

```
from .food import Food
```

```
WHITE = (255, 255, 255)
```

```
GREEN = (0, 200, 0)
```

```
RED = (220, 60, 60)
```

```
class GameEngine:
```

```
    def __init__(self, width, height):
```

```
        self.width = width
```

```
self.height = height
self.cell_size = 20

self.grid_width = width // self.cell_size
self.grid_height = height // self.cell_size
```

Save with:

</> Plain text

Ctrl + S

Test Task 3

Run:

</> Bash

```
python main.py
```

Let the snake die.

You should see:

</> Plain text

```
GAME OVER
Final Score: 0
1 - Easy
2 - Medium
3 - Hard
4 - Exit
```

Press 1. It should restart.

Then let it die again and press 2, then 3. All should restart.

Press 4 to exit.

Commit Task 3

Once tested:

</> Bash

```
git add game/game_engine.py
```

</> Bash

```
git commit -m "Add replay difficulty options"
```

Stop here after the commit.

STEP 14: Task 4, Sound Feedback

Now we add sound for:

- eating food
- Game Over

First modify **main.py**

At the very top, change:

</> Python

```
pygame.init()
```

to:

</> Python

```
pygame.mixer.pre_init(44100, -16, 1, 512)
pygame.init()
```

Then we modify **game_engine.py**

Add these imports at the top:

</> Python

```
import math
from array import array
```

Then inside **GameEngine**, immediately after **__init__** starts, add:

</> Python

```
if not pygame.mixer.get_init():
    pygame.mixer.init()
```

```
self.eat_sound = self.create_tone(800, 0.08)
self.game_over_sound = self.create_tone(220, 0.25)
```

Add this method inside the class:

</> Python

```
def create_tone(self, frequency, duration):
    sample_rate = 44100
    samples = array("h")

    for i in range(int(sample_rate * duration)):
        value = int(
            16000 * math.sin(
                2 * math.pi * frequency * i / sample_rate
            )
        )
        samples.append(value)

    return pygame.mixer.Sound(buffer=samples)
```

In `update()`, change wall collision from:

</> Python

```
self.game_over = True
return
```

to:

</> Python

```
self.game_over = True
self.game_over_sound.play()
return
```

Do the same for **self collision**.

In food collision, after:

</> Python

```
self.score += 1
```

add:

</> Python

```
self.eat_sound.play()
```

Save everything.

Test

Run:

</> Bash

```
python main.py
```

Eat food → you should hear a short beep.

Hit wall/self → you should hear a different beep.

Commit Task 4

</> Bash

```
git add game/game_engine.py main.py
```

</> Bash

```
git commit -m "Add sound feedback"
```

STEP 15: Push everything

Now run:

</> Bash

```
git push origin main
```

Your fork should now contain all four task commits.

Check GitHub and verify the commit history shows approximately:

</> Plain text

Add sound feedback
Add replay difficulty options
Implement game over screen
Fix snake reverse direction collision

STEP 16: AFTER video 🎥

Now record your **10-second AFTER video**.

Show the finished game doing something useful, preferably:

Game → gameplay → Game Over → replay menu → restart

This is much stronger evidence than simply showing the title screen.

Save it alongside your before video:

🔗 Plain text

```
videos/  
├─ before.mp4  
└─ after.mp4
```

The assignment explicitly requires both before and after videos.

 Lab_4_VibeCoding_Student_handout

Final deliverables

Your handout says the Lab-4 folder needs only:

🔗 Plain text

```
Lab-4/  
├─ before.mp4  
├─ after.mp4  
├─ Updated Code  
└─ Chat History.pdf
```

Those are the required deliverables.

 Lab_4_VibeCoding_Student_handout

Do Task 3 now, test it, commit it, then Task 4, test it, commit it. Don't push until both commits are done.

for step 14 give entire new updated file i have vvvv less time

Yes bro. ⚡ Skip individual edits. Replace the entire files. This is the fastest way.

STEP 14A: Replace ENTIRE `game/game_engine.py`

Open:

⌕ Plain text

game/game_engine.py

Press **Ctrl + A** and paste this:

⌕ Python

```
import math
from array import array

import pygame

from .snake import Snake
from .food import Food

WHITE = (255, 255, 255)
GREEN = (0, 200, 0)
RED = (220, 60, 60)

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.cell_size = 20
```

Press **Ctrl + S**.

STEP 14B: Replace ENTIRE `main.py`

Open:

⌕ Plain text

main.py

Press **Ctrl + A** and paste:

</> Python

```
import pygame
from game.game_engine import GameEngine

# Initialize pygame with audio support
pygame.mixer.pre_init(44100, -16, 1, 512)
pygame.init()

# Screen dimensions
WIDTH, HEIGHT = 600, 600
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Snake - Pygame Version")

# Colors
WHITE = (255, 255, 255)
BLACK = (0, 0, 0)
```

Press **Ctrl + S**.

STEP 14C: Test

Run:

</> Bash

```
python main.py
```

Test quickly:

1. **Eat food** → should make a short beep 🔊
2. **Hit wall** → should make a different beep 🔊
3. **Game Over menu appears:**

</> Plain text

```
GAME OVER
Final Score: 0
1 - Easy
2 - Medium
```

3 - Hard
4 - Exit

4. Press **1** → restarts Easy.

5. Press **4** → exits.

If all works

Immediately run:

</> Bash

```
git add game/game_engine.py main.py
```

then:

</> Bash

```
git commit -m "Add sound feedback"
```

then:

</> Bash

```
git push origin main
```

After that, **record your 10-second AFTER video** showing the finished game. 🎥

Then we only have the **Lab-4 folder + chat-history PDF** left.

yeah done

Perfect bro. 🔥 **Coding part is DONE.** Now we just finish the submission package.

STEP 15: Prepare the **Lab-4** folder

Create a folder called:

</> Plain text

```
Lab-4
```

Inside it, keep exactly:

</> Plain text

Lab-4

├─ before.mp4

├─ after.mp4

├─ 45_snake_updated.zip

└─ Chat_History.pdf

The handout requires **before video, after video, updated code, and exported chat history as a DOC/PDF.**  Lab_4_VibeCoding_Student_handout

Updated code ZIP

Go to the folder containing:

</> Plain text

45_snake

Right-click **45_snake** → **Send to** → **Compressed (zipped) folder**

Rename it:

</> Plain text

45_snake_updated.zip

Put that ZIP inside **Lab-4** .

Chat history

Since the assignment specifically asks for the **chat history exported as a DOC/PDF**, export this ChatGPT conversation using the ChatGPT export/share option available in your interface and save it as:

</> Plain text

Chat_History.pdf

Put that inside **Lab-4** .

Final check

Before submitting, your folder should look like:

</> Plain text

Lab-4

```
|  
├─ before.mp4      ✓  
├─ after.mp4       ✓  
├─ 45_snake_updated.zip ✓  
└─ Chat_History.pdf ✓
```

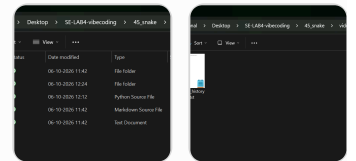
Do not add SQL, screenshots, PRs, or extra files unless your instructor separately asked for them.

Also make sure your GitHub fork has the 4 separate commits and that you **did not** create a PR to

SETAPESU26 .

 Lab_4_VibeCoding_Student_handout

That's the finish line. 🏁 



its ok no?

Almost ✓ The videos are perfectly placed:

 Plain text

videos/

```
├─ before.mp4 ✓  
└─ after.mp4  ✓
```

But one thing is NOT correct yet:

 Plain text

chat_history link.txt ✗

The handout specifically requires the **chat history exported as a DOC/PDF**, not a text file containing a link.

 Lab_4_VibeCoding_Student_handout

So your final **Lab-4** submission should contain:

 Plain text

Lab-4/

```
├─ before.mp4  
└─ after.mp4
```

└─ 45_snake_updated.zip
└─ Chat_History.pdf

Your actual `45_snake` project already contains the updated code, so zip that folder as:

`45_snake_updated.zip`

Then replace:

`chat_history link.txt` → `Chat_History.pdf`

That's the only thing left to fix. 🤖

Do NOT submit the `chat_history link.txt` as the chat-history deliverable.



see i dont hv to upload code ig just the gpr link and other delievrarables so this isok?

Yes, the GitHub code part looks good ✅. Your screenshot shows:

- Your fork is **4 commits ahead** of `SETAPESU26/45_snake:main` ✅
- The latest commit is **Add sound feedback** ✅
- You did **not create a PR** ✅
- Your updated code is pushed to your fork ✅

But one important distinction:

The handout says the **Lab-4 folder itself must contain/upload:**
videos (before + after), updated code, and chat history as DOC/PDF.

 Lab_4_VibeCoding_Student_handout

So:

Your current status

GitHub project repo: ✅ DONE

Before video: ✅

After video: ✅

Updated code: ✅ available in GitHub

Chat history PDF/DOC: ❌ still needs to be provided as a PDF/DOC, not `chat_history link.txt`

So you **do not need to separately upload the code as a ZIP** *if your submission system accepts the GitHub repo link as the updated-code reference*. But the handout itself does not explicitly say "submit a GitHub link instead of the code," so don't assume that unless your submission page says so.

The safest final submission

Give your instructor/submission portal:

GitHub repo: your `45_snake` fork link

Lab-4: `before.mp4` , `after.mp4` , `Chat_History.pdf`

And make sure the **chat history is actually a PDF/DOC**. That's the one thing I would still fix. 🤖

Your code repository screenshot itself looks ready.